

Evaluación de etiquetadores morfosintácticos para el español

Gerard Mas Mollà
Miguel García Bohigues

October 25, 2022

Contents

1	Introducción	3
2	Etiquetador HMM con variación de etiquetas	4
3	Etiquetador HMM con entrenamiento incremental	5
4	Suavizado por sufijos para TNT	6
5	Otros etiquetadores	8
6	Evaluación de la herramienta Spacy	9
6.1	Instalación	9
6.2	Uso de la herramienta	10
7	Preguntas	10

1 Introducción

En el presente trabajo, se ha realizado un estudio de algunos de los diferentes etiquetadores morfosintácticos que la biblioteca *NLTK* de *Python* nos proporciona. La evaluación de los etiquetadores se ha realizado sobre el corpus en español *cess_esp* y la unidad de medida utilizada para poder comparar los diferentes sistemas ha sido la precisión que reflejan los sistemas. Esta la hemos representado junto a su correspondiente intervalo de confianza al 95%.

Siguiendo este criterio, se han realizado las cuatro tareas obligatorias y la quinta opcional. Correspondiéndose con:

- La evaluación de un etiquetador basado en modelos ocultos de Markov comparando su rendimiento con el juego completo de categorías y con un subconjunto de las mismas.
- La evaluación de un etiquetador basado en modelos ocultos de Markov en función del número de muestras de aprendizaje de las que dispone para entrenar.
- La evaluación de un etiquetador **TNT** comparando si la utilización de un suavizado basado en *Affix tagger* mejora las prestaciones obtenidas por el clasificador sin suavizado.
- La evaluación de otros etiquetadores, tales como el de *Brill*, *CRF* o *Perceptron*.
- La evaluación de la herramienta de libre distribución *Spacy* sobre el fichero de corpus proporcionado.

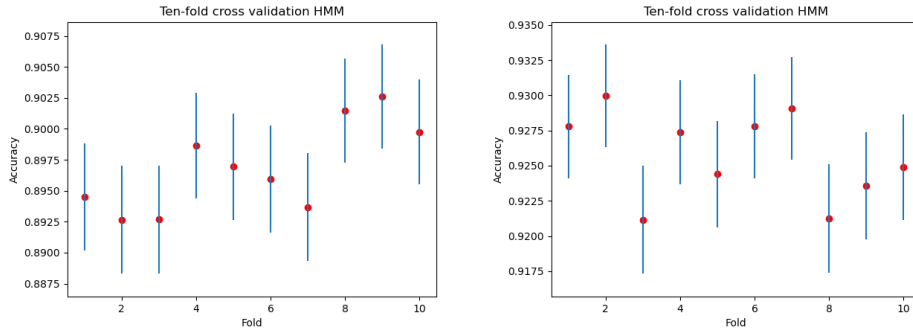
Todo el desarrollo se ha desarrollado sobre *Python 3.10*, utilizando la distribución de *Anaconda*. Además, cada una de las tareas se ha dividido en un fichero *.py* diferente para facilitar su desarrollo, depuración y evaluación. Se ha hecho uso de un fichero auxiliar de utilidades para hospedar la lógica de uso recurrente en las tareas.

2 Etiquetador HMM con variación de etiquetas

En este apartado comparamos el funcionamiento de un *HMM Tagger* en función del número de etiquetas que consideremos para la clasificación. En concreto hemos presentado dos escenarios: en el primero el clasificador ha de trabajar con todo el conjunto de etiquetas que aparecen en el corpus, 289; por otro lado, hemos realizado una generalización sobre las etiquetas considerando solo los primeros 3 caracteres de las mismas, reduciendo su número hasta las 66.

La generalización de las etiquetas se ha realizado eliminando de la etiqueta cierta información morfosintáctica. En concreto, vamos a quedarnos con sólo el tipo de palabra con la que estamos tratando (nombre común, nombre propio, adjetivo...) Por ejemplo, para la palabra 'Discrepancias', su etiqueta completa es 'ncfp000' (Nombre Común Femenino Plural...) de ésta, nos vamos a quedar sólo con el segmento 'nc'. Para el caso de los verbos y los signos de puntuación vamos a operar de la misma manera, pero en lugar de quedarnos con los 2 primeros caracteres de la etiqueta, lo ampliaremos a los 3 primeros.

Como se menciona en la rúbrica publicada, se ha realizado una validación cruzada de 10 segmentos para cada uno de los casos enunciados previamente. Los resultados obtenidos se pueden observar en la figura 4 dónde mostramos la precisión de ambos sistemas junto con su intervalo de confianza.



(a) HMM 10-fold CV con todas las cate- (b) HMM 10-fold CV con un subset de 66
gorías categorías

Figure 1: Diferencia de rendimiento de un etiquetador HMM según el número de categorías

Por la diferencia de dificultad en el problema de clasificación, el etiquetador entrenado con el conjunto completo de las características ha mostrado peor rendimiento. Por su parte, el clasificador reducido, ha obtenido muy buenos resultados, casi 3 puntos de precisión más de media que el completo. En la tabla 1 se puede ver desglosados los resultados de los experimentos, junto a la media obtenida a lo largo de las 10 validaciones cruzadas.

Table 1: Resultados de las validaciones para ambos sistemas.

HMM [$\ C\ = 289$]		HMM [$\ C\ = 66$]	
Accuracy	IC	Accuracy	IC
0.89450	± 0.00433	0.92778	± 0.00369
0.89266	± 0.00437	0.92995	± 0.00364
0.89267	± 0.00437	0.92115	± 0.00384
0.89862	± 0.00426	0.92735	± 0.00370
0.89695	± 0.00429	0.92440	± 0.00377
0.89594	± 0.00431	0.92780	± 0.00369
0.89367	± 0.00435	0.92907	± 0.00366
0.90144	± 0.00420	0.92124	± 0.00384
0.90260	± 0.00418	0.92356	± 0.00378
0.89975	± 0.00424	0.92489	± 0.00375
Average Acc.	0.89688	Average Acc.	0.925719

3 Etiquetador HMM con entrenamiento incremental

Ya hemos visto el comportamiento de un etiquetador basado en modelos ocultos de Markov en función del número de clases que consideremos como etiquetas. En este apartado vamos a evaluar el comportamiento que tiene este etiquetador en función de la cantidad de elementos con los que se entrene.

Para llevarlo a cabo vamos a partir del modelo de 66 etiquetas comentado en la sección anterior. Éste lo hemos entrenado de manera incremental con cada vez más muestras de entrenamiento, validándolo siempre contra las mismas muestras de test para poderlos comparar. Para ello, hemos segmentado el corpus en 10 partes iguales. De manera iterativa hemos entrenado y probado el modelo con una, dos... y hasta 9 partes juntas, validando el resultado con el décimo conjunto, el de test.

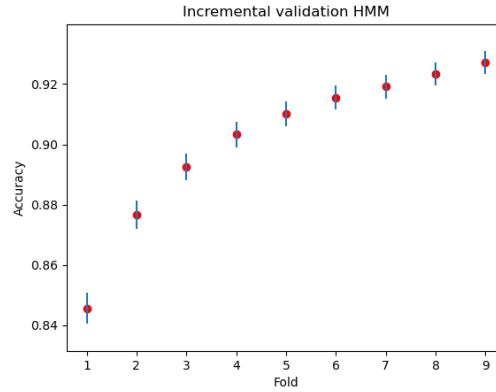


Figure 2: HMM entrenamiento incremental 10-fold

Table 2: Resultados del modelo para cada uno de los conjuntos incluidos en el entrenamiento.

N	Accuracy	IC
1	0.84550	± 0.00515
2	0.87671	± 0.00468
3	0.89242	± 0.00441
4	0.90324	± 0.00421
5	0.91011	± 0.00407
6	0.91549	± 0.00396
7	0.91918	± 0.00388
8	0.92341	± 0.00379
9	0.92720	± 0.00370

Como podemos ver en la gráfica 2, a medida que aumentamos el tamaño del conjunto de entrenamiento, el sistema es capaz de precedir con mayor precisión la categoría de las palabras, obteniendo el máximo valor de la precisión utilizando nueve de los diez conjuntos para entrenar y el restante para validar los resultados. No obstante, el resultado que obtiene no es lo suficiente alto para concluir que es mejor que cualquiera de las otras combinaciones, puesto que su intervalo de confianza se solapa con el modelo que utiliza ocho de los diez conjuntos para train. Aunque sí podemos afirmar que supera a todos los conjuntos inferiores a éste. La tabla 2 muestra los resultados en formato tabular para facilitar su análisis:

4 Suavizado por sufijos para TNT

TNT es un modelo de etiquetado de palabras que no presenta un método de suavizado para palabras que no ha visto en fase de entrenamiento. En esta sección del trabajo estudiamos la inclusión de un modelo de suavizado basado en sufijos, *Affix tagger*. En concreto hemos experimentado con diferentes longitudes para los sufijos intentando establecer qué valor es el mejor y así compararlo con el rendimiento del modelo base para ver si mejora sus prestaciones.

Para el *Affix Tagger* hemos considerado longitudes de sufijos entre uno y cuatro caracteres. Hemos tomado esta decisión porque, tras analizar el corpus, hemos visto que la longitud media de las palabras es de 4.766 caracteres y la mediana de 3.

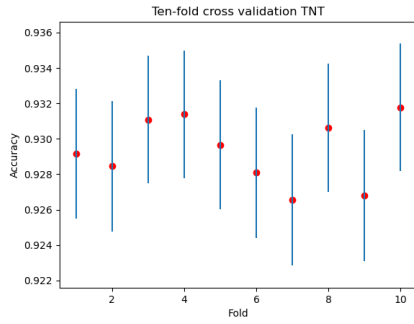
A continuación se muestran los resultados, tanto en formato tabular como gráficamente. En ellos se pueden observar las validaciones cruzadas para cada uno de los modelos entrenados: TNT sin suavizado y TNT con suavizado por *Affix tagging* para longitudes de uno a cuatro.

Como podemos ver en la tabla 3, a medida que aumentamos el tamaño del sufijo, vamos obteniendo mejor valor de precisión hasta llegar a su pico para sufijos de tamaño 3. A partir de aquí se ha observado que se va reduciendo la precisión del sistema, por ello también se ha decidido no probar con valores superiores a 4. Además

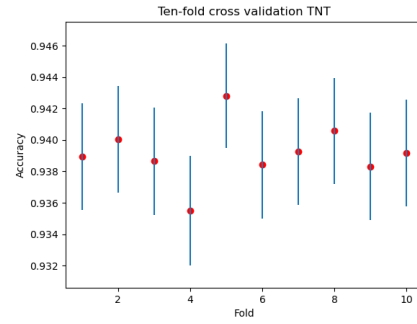
Table 3: Valores medios obtenidos de las validaciones cruzadas.

Etiquetador TNT			
Suavizado	Tam. suf.	Accuracy Mean	IC
No	-	0,90128	$\pm 0,004254$
Si	1	0,92935	$\pm 0,003655$
Si	2	0,93917	$\pm 0,003409$
Si	3	0,94707	$\pm 0,003193$
Si	4	0,94396	$\pm 0,003280$

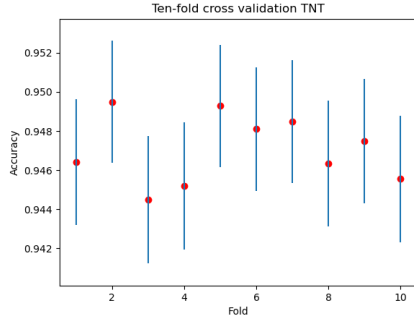
de lo comentado en el párrafo introductorio de esa sección.



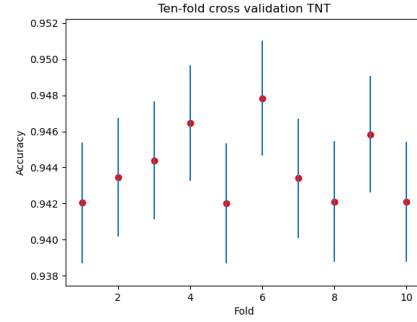
(a) 10-fold |suf|=1



(b) 10-fold |suf|=2



(c) 10-fold |suf|=3



(d) 10-fold |suf|=4

Figure 3: Validación cruzada con TNT para cada longitud del sufijo

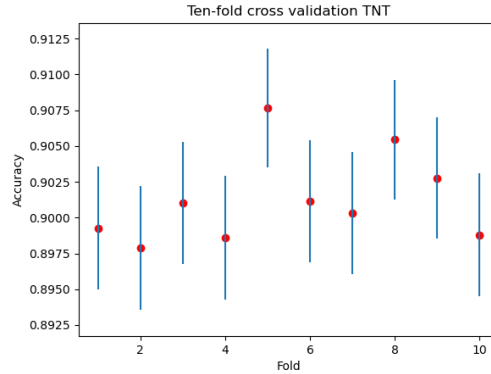


Figure 4: Validación cruzada con TNT sin suavizado

5 Otros etiquetadores

Finalmente, para acabar con la evaluación de etiquetadores para el castellano, vamos a hacer una rápida revisión de otros clasificadores disponibles en la librería *NLTK*.

Además de las aproximaciones realizadas para HMM y TNT, hemos entrenado tres clasificadores más.

El primero ha sido el etiquetador de Brill. Para ello hemos optado por utilizar 2 modelos distintos como entrada: un HMM y un modelo de unigramas. Además, para cada uno de los modelos de entrada, hemos probado tres valores diferentes de *templates*: *demo18*, que es el valor por defecto que presentaba *nlk*; *demo18+*, que añade al anterior un mecanismo *multifeature*; y *brill24*, que es el que presentó en su paper de 1995.

Además de estos, hemos desarrollado dos clasificadores más, un *CRF* y un *Perceptron*.

Table 4: Valores medios obtenidos de las validaciones cruzadas.

Modelo	Baseline model	Templates	Accuracy	IC
Brill	HMM	demo18	0.92832	± 0.00367
Brill	HMM	demo18+	0.92928	± 0.00365
Brill	HMM	brill24	0.92922	± 0.00365
Brill	Unigrama	demo18	0.90222	± 0.00423
Brill	Unigrama	demo18+	0.90317	± 0.00421
Brill	Unigrama	brill24	0.90349	± 0.00421
Perceptron	-	-	0.96089	± 0.00276
CRF	-	-	0.95517	± 0.00295
TNT	-	$ \text{suffix} =3$	0.94707	± 0.00319
HMM	-	-	0.92571	± 0.00374

Los resultados conjuntos de estos clasificadores se pueden observar en la tabla 4. Podemos ver como los etiquetadores que mejor resultado han reflejado han sido, por orden, *Perceptron* y *CRF*, con precisiones superiores al 95%, seguidos muy de cerca por el etiquetador TNT con suavizado por sufijos de tamaño 3. Podemos observar también cómo para el modelo de Brill, obtenemos mejores resultados partiendo de un etiquetador basado en modelos de markov que con un etiquetador de unigramas, independientemente de los templates. Comentar que en la tabla hemos introducido para los modelos HMM y TNT, los mayores valores que hemos obtenido a lo largo del estudio que hemos realizado sobre ellos.

6 Evaluación de la herramienta Spacy

Para esta tarea, se ha decidido evaluar el funcionamiento de la herramienta Spacy. Spacy es una librería open-source de python para procesamiento del lenguaje natural avanzado desarrollada por los fundadores de Explosion AI que destaca por su facilidad de uso y su amplia oferta de modelos preentrenados. Así como NLTK está fuertemente extendido en contextos educativos, Spacy ha sido ampliamente adoptada en contextos industriales. Tiene soporte para modelos neuronales y cuenta con una amplia y detallada documentación en su página web¹

En este caso, se utilizará Spacy para etiquetar morfosintácticamente el archivo *Alicia.txt* utilizando un modelo preentrenado.

6.1 Instalación

Spacy se puede instalar a través del package manager de python, **pip** (Preferred Install Program) a través de un simple comando de terminal:

```
1 pip install spacy
```

Este comando hará que **pip** se encargue de instalar la librería e instalar los paquetes necesarios para solucionar los conflictos de dependencias que puedan haber.

Adicionalmente, para instalar el modelo preentrenado que vamos a usar para etiquetar el corpus dado, debemos ejecutar otra orden de terminal:

```
1 python -m spacy download es_core_news_$sz
```

Donde $\$sz = \{ \text{sm}, \text{md}, \text{lg} \}$ determina el tamaño del modelo que se va a descargar: "small", de 12MB, "medium", de 40MB o "large", de 541MB. Además, Spacy también ofrece un modelo neuronal: **es_dep_news_trf**, que es un modelo basado en la arquitectura transformer que ocupa 391MB.

Tras ejecutar estas dos ordenes, ya se puede utilizar la herramienta Spacy y los modelos preentrenados nombrados.

¹<https://spacy.io/>

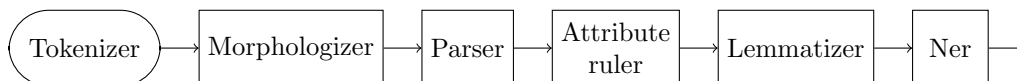


Figure 5: Pipeline del etiquetador de Spacy

6.2 Uso de la herramienta

Como se ha comentado, se ha optado por emplear modelos preentrenados para etiquetar el archivo *alicia.txt*. Para utilizar estos modelos preentrenados, basta con importar la librería y cargar el modelo:

```

1 import spacy
2 nlp = spacy.load("model")

```

De esta forma, `nlp` actúa como una función que usa el modelo para etiquetar un texto pasado por argumento:

```

1 result = nlp("Frase a etiquetar")

```

Almacenando en `result` el resultado del etiquetado. En esta variable encontramos, para cada token, de la frase etiquetada, la palabra, su forma lematizada, su etiqueta UPOS, la etiqueta POS detallada, la función que cumple en la frase... resultado de pasar la frase por el pipeline del modelo, mostrada en la figura 5.

A continuación, utilizando las instrucciones explicadas, se ha pasado el archivo *alicia.txt* por el etiquetador, obteniendo las etiquetas que se muestran en la figura 6

7 Preguntas

- ¿Qué valor es el ideal para el tamaño del sufijo en el suavizado de *Affix tagger* aplicado a TNT? ¿Por qué crees que es esto?
El valor que mejor resultado ha reflejado es aquel que presentaba 3 como tamaño del sufijo. Una hipótesis de por qué puede ser esto así, es porque el tamaño medio de las palabras en el corpus es de aproximadamente 4,75 caracteres.
- ¿Qué cuatro conjuntos de *templates* vienen incluidos en el módulo *Brill* y que podemos utilizar directamente para entrenar el etiquetador?
Los conjuntos de *templates* son: demo18, demo18plus, brill24, fntbl37.

```

Small model
A/ADP través/NOUN de/ADP la/DET tarde/NOUN color/NOUN de/ADP oro/NOUN
el/DET agua/NOUN nos/PRON lleva/VERB sin/ADP esfuerzo/NOUN por/ADP nuestra/DET parte/NOUN ,/PUNCT
pues/SCONJ los/DET que/PRON empujan/VERB los/DET remos/NOUN
son/AUX unos/DET brazos/NOUN infantiles/ADJ
que/PRON intentan/VERB ,/PUNCT con/ADP sus/DET manitas/NOUN
guiar/VERB el/DET curso/NOUN de/ADP nuestra/DET barca/NOUN ./PUNCT

medium model
A/ADP través/NOUN de/ADP la/DET tarde/NOUN color/NOUN de/ADP oro/NOUN
el/DET agua/NOUN nos/PRON lleva/VERB sin/ADP esfuerzo/NOUN por/ADP nuestra/DET parte/NOUN ,/PUNCT
pues/SCONJ los/DET que/PRON empujan/VERB los/PRON remos/VERB
son/AUX unos/DET brazos/NOUN infantiles/ADJ
que/PRON intentan/VERB ,/PUNCT con/ADP sus/DET manitas/NOUN
guiar/VERB el/DET curso/NOUN de/ADP nuestra/DET barca/NOUN ./PUNCT

large model
A/ADP través/NOUN de/ADP la/DET tarde/NOUN color/NOUN de/ADP oro/NOUN
el/DET agua/NOUN nos/PRON lleva/VERB sin/ADP esfuerzo/NOUN por/ADP nuestra/DET parte/NOUN ,/PUNCT
pues/SCONJ los/DET que/PRON empujan/VERB los/DET remos/VERB
son/AUX unos/DET brazos/NOUN infantiles/ADJ
que/PRON intentan/VERB ,/PUNCT con/ADP sus/DET manitas/NOUN
guiar/VERB el/DET curso/NOUN de/ADP nuestra/DET barca/NOUN ./PUNCT

Transformer model
A/ADP través/NOUN de/ADP la/DET tarde/NOUN color/NOUN de/ADP oro/NOUN
el/DET agua/NOUN nos/PRON lleva/VERB sin/ADP esfuerzo/NOUN por/ADP nuestra/DET parte/NOUN ,/PUNCT
pues/SCONJ los/DET que/PRON empujan/VERB los/DET remos/NOUN
son/AUX unos/DET brazos/NOUN infantiles/ADJ
que/PRON intentan/VERB ,/PUNCT con/ADP sus/DET manitas/NOUN
guiar/VERB el/DET curso/NOUN de/ADP nuestra/DET barca/NOUN ./PUNCT

```

Figure 6: Etiquetado de los cuatro modelos distintos sobre el primer párrafo del corpus Alicia